

DeepInv iterative optimization refactor: diagnosis

Draft 0.2 for maintainer discussion

Audited branch: refactor/optim at [3f832a7c2](#)

Scope: diagnosis only; architecture and migration are deferred.

Main issues:

- execution flow
- responsibility and ownership overlap
- no contract for the internal state
- optional behavior embedded in and polluting the common execution path
- small optimization algorithms implemented through separate execution loops

1. Execution flow

The execution flow is indirect and difficult to understand. The depth is not the issue by itself, some components are going back-and-forth across all these layers. Here's the typical flow:

```
PGD public constructor
  -> FixedPoint.forward
    -> FixedPoint.single_iteration
      -> PGDIteration          orchestration
        -> fStepPGD
        -> gStepPGD

least_squares
  -> CG / BiCGStab / LSQR / MINRES
  -> solver-owned loop, state and stopping
```

The main loop is in `FixedPoint.forward`, but its behavior is partly dictated by `BaseOptim`. It is very hard to change one part of the orchestration without breaking everything.

2. Shared ownership across layers

The different layers of the execution flow above might share common responsibilities and own similar logic and states.

Initialization, objective evaluation, internal states are typically modified and used in all of the layers from `FixedPoint` to a step.

Ownership also seems a bit arbitrary:

- `BaseOptim` owns configuration, initialization, monitoring, trainable parameters, gradient modes, and DEQ behavior.
- `FixedPoint` owns the loop, callback order, acceleration, backtracking, history, and stopping.
- `OptimIterator` owns the update, but also objective evaluation and step-size relaxation

3. Unclear state contract

The internal state is typically `{"est": tuple, "cost": value}`, but algorithms use one, two, or three estimates and may add fields such as `"it"`.

Required fields, ownership, mutation rules, and feature compatibility are not expressed by a contract (i.e typed params, attributes, return types, etc) anywhere.

Having a common internal dictionary with minimum base fields would already be an improvement.

Some other examples:

- common code assumes that the reconstructed image is always `x["est"][0]` but it is barely written anywhere
- `x_prev = x` assumes iterators return fresh state instead of mutating state in place, which is also barely written

4. Optional behavior in the common path

The common loop mixes mandatory and optional behaviors. The implementation is obscured by all the optional stuff in the middle, similarly to the `Trainer`.

Metrics, early stopping, backtracking, Anderson acceleration, progress display, unfolding, and DEQ are optional, but take most of the implementation.

This is amplified by the unclear ownership and responsibility: these optional parts are polluting both `BaseOptim` and `FixedPoint`

5. Fragmented optimization workflows

Linear solvers use a completely separate execution flow. This is not unique to them: `optim.utils.gradient_descent`, EPLL and spectral methods also own their loop, state and stopping.

There is no lightweight common execution contract, so new algorithms must either fit the current hierarchy or create another isolated loop.